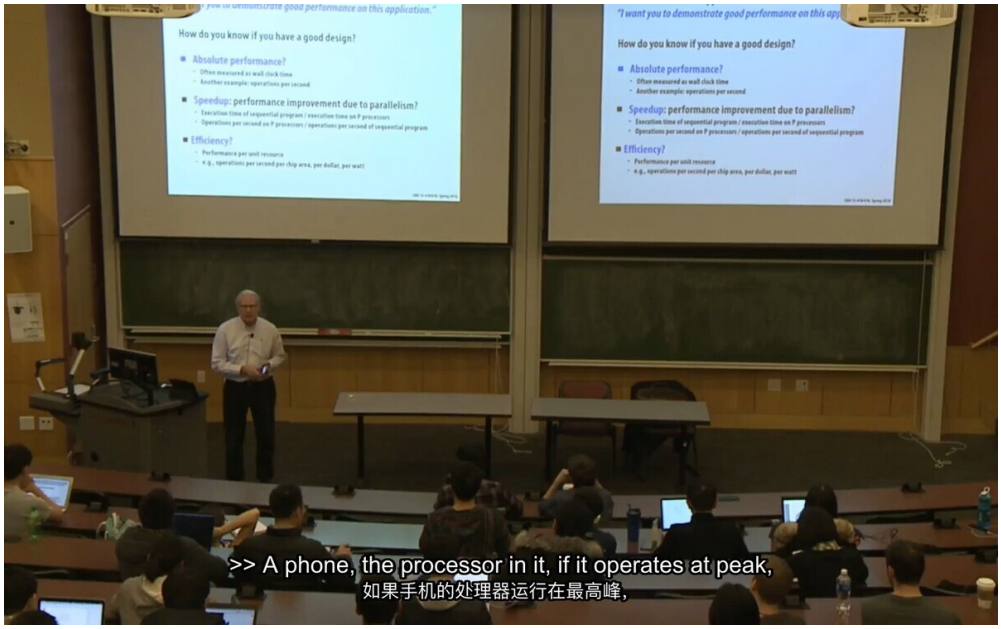
课程笔记

# 并行程序的算例分析：加速比的陷阱与扩展性约束

CMU 15-418/618 并行计算机架构与编程 · Lecture 10（Spring 2018）

lecture-to-notes

2026 年 10 月 28 日




**视频作者/频道**：CMU 15-418/618（B 站搬运：alexphil）
**发布日期**：2018 年春季学期
**视频时长**：1 小时 7 分 9 秒
**视频链接**：`https://www.bilibili.com/video/BV1TV4y1e7s5/`

# 目录

**1 加速比这个指标本身有问题** **2**
1.1 先从功耗说起 . . . 2
1.2 一个用来检验测量方法的例子 . . . 2
1.3 本章小结 . . . 3

**2 加速比的两个陷阱** **3**
2.1 第一个陷阱：基线和谁比 . . . 3
2.2 第二个陷阱：问题太小 . . . 4
2.3 换个问题规模，结论完全不同 . . . 5
2.4 本章小结 . . . 6

**3 超线性加速说明了什么** **6**
3.1 缓存合并带来的超线性 . . . 6
3.2 另一个真实的超线性来源 . . . 6
3.3 扩展性不是单一概念 . . . 7
3.4 本章小结 . . . 7

**4 强扩展与弱扩展** **7**
4.1 两种术语 . . . 7
4.2 看懂架构的可扩展设计 . . . 8
4.3 本章小结 . . . 9

**5 三种约束下的定量分析** **9**
5.1 三维参数：扩展本身是多变量的 . . . 9
5.2 约束一：问题约束（经典加速比） . . . 10
5.3 约束二：时间约束 . . . 11
5.4 约束三：内存约束 . . . 12
5.5 三种约束的排序 . . . 13
5.6 本章小结 . . . 15

**6 真实场景中的扩展与模拟方法** **15**
6.1 同一应用，不同约束 . . . 15
6.2 本章小结 . . . 16

**7 用模拟评估架构** **16**
7.1 为什么要靠模拟 . . . 16
7.2 跟踪驱动的两个风险 . . . 17
7.3 本章小结 . . . 18

**8 总结与延伸** **18**

# 1 加速比这个指标本身有问题

这一讲从一次质疑开始：怎样才算”在这个应用上取得了良好性能”？讲者指出，回答这个问题比想象中难，因为最常用的指标——加速比——在多种情况下会给出误导性的结论。

## 1.1 先从功耗说起

讲者在开场提到一个与性能常被混为一谈的量：手机里的处理器若跑到峰值要烧掉约 5 瓦，放在口袋里会让你觉得腿很热。因此它典型运行功耗只有 100 到 200 毫瓦。

这解释了为什么芯片设计要做得如此复杂：很大一部分动机不是让它在最高速度下表现更好，而是让它能在低功耗下更高效地运行。

这个落差可以用两个量的比值来表达：

$$R_{\text{power}} = \frac{P_{\text{peak}}}{P_{\text{typical}}}$$

- $P_{\text{peak}}$：芯片能维持的峰值功耗，本例约 5 瓦；
- $P_{\text{typical}}$：实际持续运行的功耗，约 100 到 200 毫瓦；
- $R_{\text{power}}$：两者之比，可达数十倍。

峰值只能短暂维持，因此持续性能由 $P_{\text{typical}}$ 决定，而非峰值算力。

**移动端的性能目标是每瓦产出而非峰值**

在功耗受限的设备上，峰值性能往往无法长时间维持。设计的重心因此从”最快”转向”每单位能耗完成多少工作”。评估这类系统时，用峰值算力比较会得出与实际体验完全不同的结论。

## 1.2 一个用来检验测量方法的例子

讲者回顾上一讲的粒子分箱问题：一批分布在空间中的粒子，要按位置分到各个网格里，好让后续处理能够局限在局部。

他列出三种并行实现。第一种是遍历所有粒子，判断它属于哪个格子，然后追加到一个列表上——若这个列表是全局的，就有跨处理器的争用，必须同步，代价可能很高。第二种是遍历所有格子，对每个格子再看一遍所有粒子，挑出属于它的。第三种最复杂：每个处理器先为自己处理的那批粒子建立局部数据结构，再以某种方式合并成描述整个系统的全局结构。

讲者点出一个很好的检验标准：把 $P$ 设为 1，三种实现都应当退化为串行算法。第一种不需要锁；第二种退化为对每个格子跑一遍；第三种不需要排序与合并，直接得到想要的结构。

这个检验可以写成一条约束：

$$T_{\parallel}(1) \;\approx\; T_{\text{serial}}$$

- $T_{\parallel}(1)$：并行实现在 1 个处理器上的耗时；
- $T_{\text{serial}}$：纯粹的串行实现耗时；
- 若两者相差很大，说明并行实现在单线程下仍背负着并行开销。

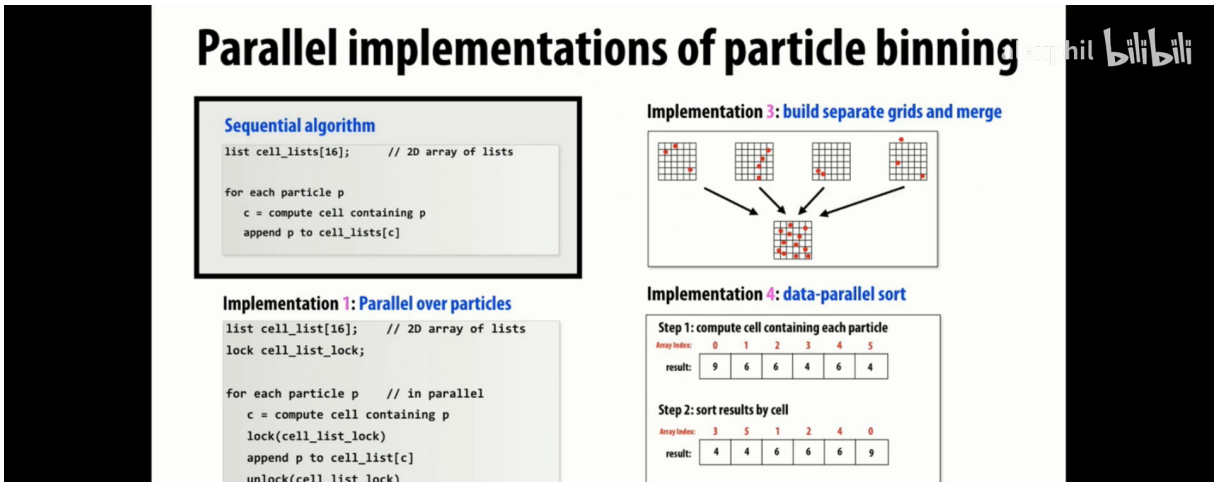



图 1: 粒子分箱的三种并行实现：第三种先建局部结构再合并[1]

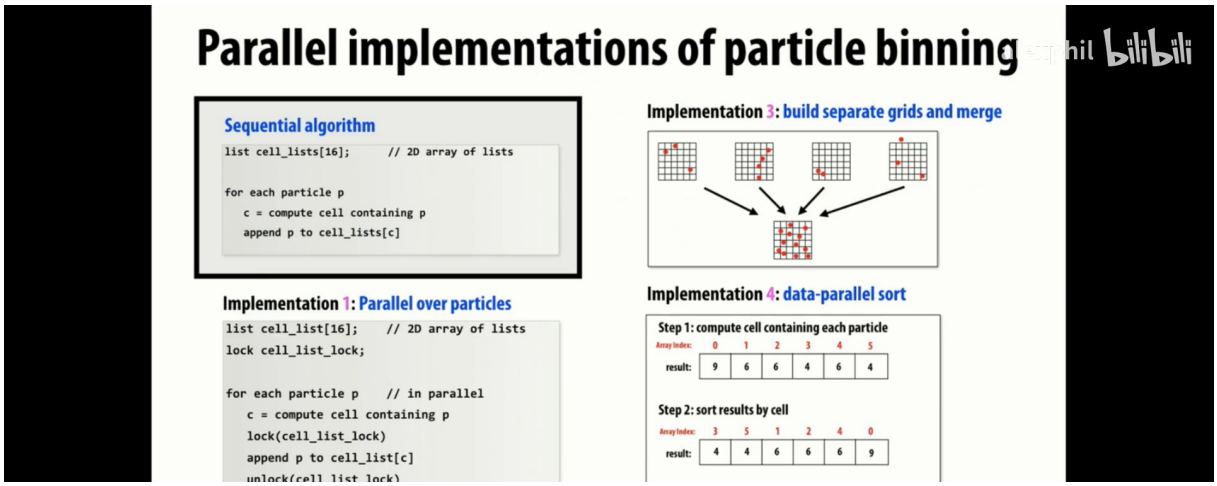



图 2: 粒子分箱的并行实现与串行基线对比[2]

### 1.3 本章小结

加速比的问题不在于这个比值本身，而在于分子分母是否可比：基线要选最优串行实现，问题规模要足够大到能喂饱所有处理器。两个陷阱方向相反，一个高估收益、一个低估收益。

在具体分析这些陷阱之前，先看清它们各自的表现形式与成因。

## 2 加速比的两个陷阱

### 2.1 第一个陷阱：基线和谁比

讲者说这里有一个在论文里常见的错误。如果拿第一种实现和”它自己的单处理器版本”比——那个版本里仍然调用着锁的接口，只是没有争用——加速比看起来不会完美，但也不难看。

[1] 视频画面时间区间：00:01:30–00:01:45。若为裁剪图，时间区间仍对应原视频画面。

[2] 视频画面时间区间：00:02:10–00:02:25。若为裁剪图，时间区间仍对应原视频画面。

但他的判断是斩钉截铁的：如果在真实世界里把它和上面那个最优的串行版本对比，结果会很难看，因为锁很贵，争用程度高的锁尤其贵。

由此得到这条原则：

**加速比必须对照最优串行实现**

分母应当是你能写出的最好的串行代码，而不是并行代码去掉并行之后的版本。后者往往保留了为并行而付出的开销（锁、原子操作、额外的数据结构），用它作基线会系统性高估并行收益。这种错误在学术论文中也并不少见。

## 2.2 第二个陷阱：问题太小

第二个问题更普遍。讲者用海洋模拟的实测数据说明：把同一份问题分别跑在 1 个和 $P$ 个处理器上，随处理器增加，加速比先是不错地上升，到 16 个处理器时几乎不再提升，到 32 个时反而开始变差。

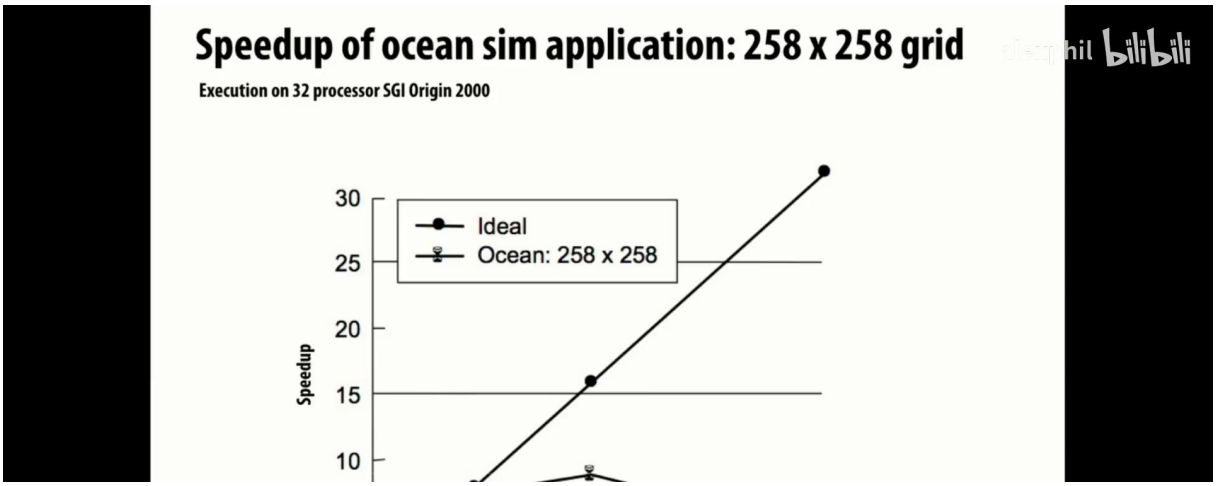



图 3: 海洋模拟在 $258 \times 258$ 网格上的加速比：16 核后停滞，32 核反而下降[3]

原因是问题太小。当处理器增加到一定程度，每个处理器分到的工作已经不足以让它保持忙碌。

讲者用上一讲的结论解释这件事。这个例子采用二维分块，算术强度为：

$$\text{AI} \propto \frac{N}{\sqrt{P}}$$

- $N$：网格的线性边长，网格共 $N^2$ 个元素；
- $P$：处理器数量；
- 结果与 $N/\sqrt{P}$ 成正比，即指数为 $-0.5$。

$P$ 增大时算术强度按 $\sqrt{P}$ 下降。讲者说这不是灾难——不是随 $P$ 线性下降——但迟早会追上来，让你到达”算术做得不够、通信占比过高、于是拿不到任何性能提升”的境地。

[3] 视频画面时间区间：00:04:30–00:04:45。若为裁剪图，时间区间仍对应原视频画面。

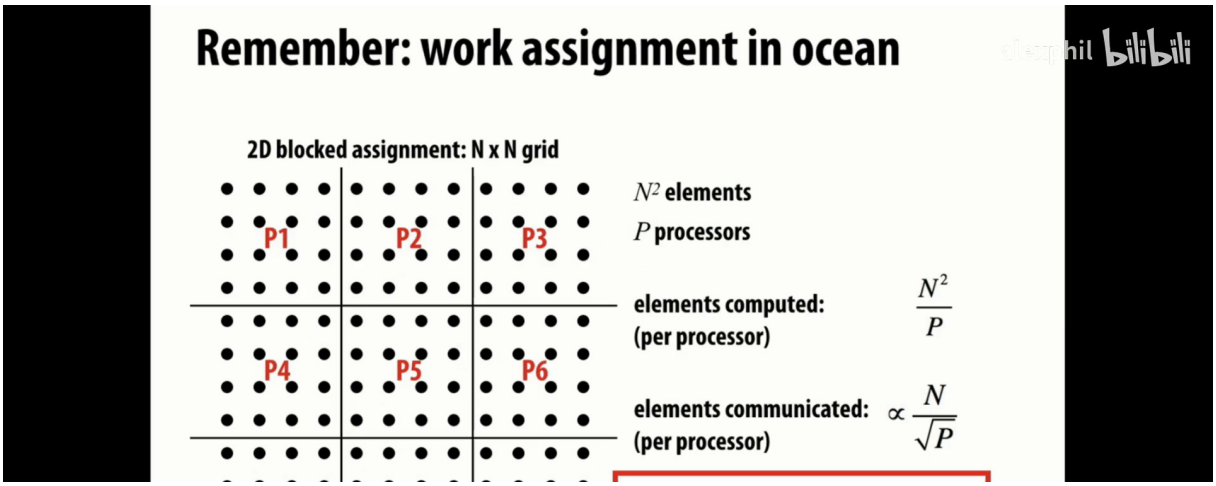



图 4: 回顾：海洋模拟采用二维分块分配，算术强度随处理器数下降[4]

## 2.3 换个问题规模，结论完全不同

讲者接着做了一个对照实验，这是本讲最有说服力的地方。

之前展示的只是 258 × 258 的网格——他纠正说其实更糟，是 130 × 130，对 32 个处理器而言太小了。而换成 1026 × 1026 的网格，同一份代码在 32 个处理器上能达到 25 倍的加速。

**扩展性结论必须绑定问题规模**

同一个实现在 130 × 130 网格上 32 核反而变慢，在 1026 × 1026 上却达到 25 倍加速。因此”这个程序能扩展到多少核”这个说法没有意义，必须补上问题规模。报告加速比时省略规模，等于给出了一个无法复现的数字。

讲者的结论是：在较小规模下加速比甚至负增长，恰恰说明问题不够大、留不住这么多处理器，越来越多的时间浪费在通信上。

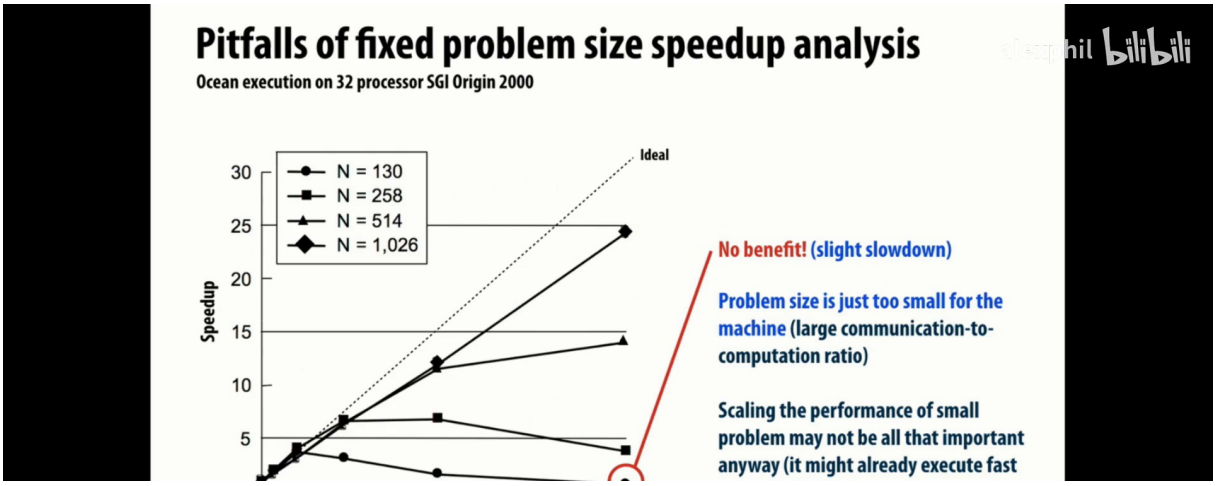



图 5: 固定问题规模加速比分析的陷阱：小网格上曲线明显下弯[5]

[4] 视频画面时间区间：00:06:30–00:06:45。若为裁剪图，时间区间仍对应原视频画面。

[5] 视频画面时间区间：00:07:00–00:07:15。若为裁剪图，时间区间仍对应原视频画面。

### 2.4 本章小结

加速比这个指标有两个必须警惕的陷阱：分母用了带并行开销的伪串行版本，以及问题规模太小以至于每一核都喂不饱。前者会高估收益，后者会低估收益。两个陷阱方向相反，说明单一加速比数字本身并不足以描述”性能好不好”。

这也引出本讲的主线：与其固定问题看加速，不如研究问题规模应如何随资源一起变化。

## 3 超线性加速说明了什么

上一节的问题规模太小。讲者随即给出反方向的另一个异常：有些情况下性能随问题规模增长得比线性还快。

### 3.1 缓存合并带来的超线性

讲者请学生猜原因，答案很快给出：缓存。

多加一个处理器，系统就多出一块缓存。而这个问题的并行性很好，可以干净地切分开。当你把问题切成 $P$ 块，每一块的工作集，也就是每个处理器需要访问的数据量，可能刚好落进了某一级缓存的容量之内。

于是每个处理器都从被缓存覆盖的状态下工作，而单处理器版本的数据量远超缓存容量，全程在访问主存。两相对比，加速比就超过了 $P$。

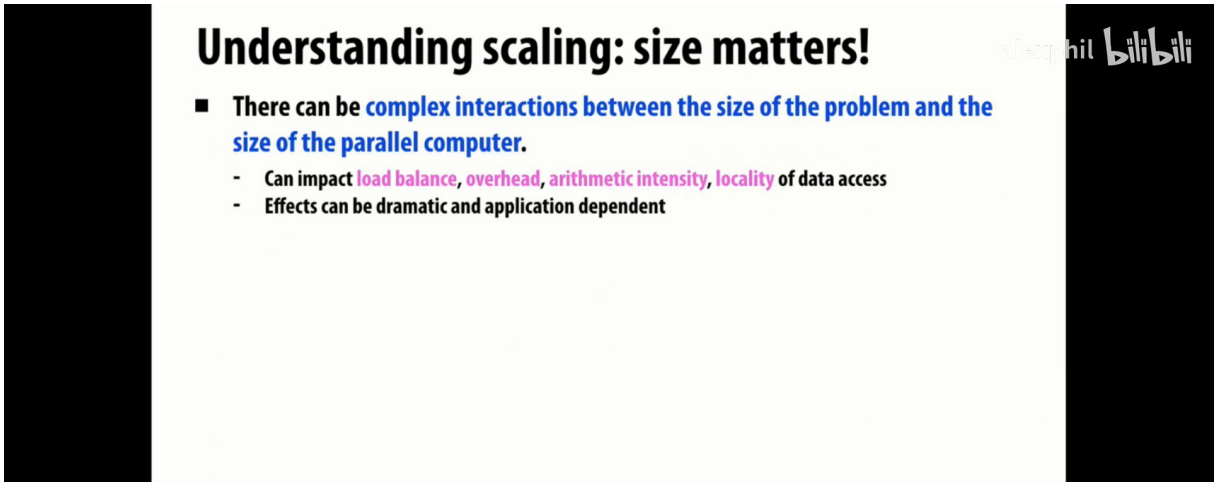



图 6: 理解扩展性：问题规模与其他因素之间存在复杂相互作用[6]

**超线性加速通常意味着基线选得不合适**

加速比超过 $P$ 在数学上不应发生在同一台机器上，因为它暗示每个处理器都比单处理器时更快。真实原因几乎总是”单处理器版本装不进缓存”。因此看到超线性时，应当先怀疑基线问题，而不是庆祝算法优越。这类数字如果出现在论文里，通常说明对照设置不公平。

### 3.2 另一个真实的超线性来源

讲者补充了一种确实存在的例外：组合搜索，例如布尔可满足性求解。

[6] 视频画面时间区间：00:10:00–00:10:15。若为裁剪图，时间区间仍对应原视频画面。

这类算法的特点是并行地尝试多个不同方向，任意一个成功就停下。同时跑的方向越多，找到解的时间就越可能缩短。这种超线性是算法性质带来的，不是测量假象。

**搜索类问题的超线性来自停止条件**

当算法满足”任一分支成功即完成”时，并行版本相当于对解空间做多重采样，期望耗时会下降得比 $1/P$ 更快。这与网格计算的性质完全不同——后者的每份工作都是必需的，无法被”跳过”。

### 3.3 扩展性不是单一概念

讲者由此提出一个总的观察：当你说”我要把问题扩大若干倍”时，即使这句话本身代表什么都不是简单的事。负载均衡、开销、算术强度、局部性这些因素都会以不同方式影响性能。改变问题或改变处理器数量，得到的关系可能非常复杂。

于是可以这样概括前面的两个例子：如果只固定一个问题问”快了多少”，那么问题太小或太大都可能让你量不出任何有意义的东西。

**”把问题放大”本身需要定义**

扩大网格边长、增加时间步数、增加粒子数，三者对性能的影响并不相同。同一份代码在”边长翻倍”与”迭代次数翻倍”下的算术强度变化趋势可能相反。因此讨论扩展性之前，必须先明确是哪个维度在变。

### 3.4 本章小结

超线性加速多数是缓存合并造成的测量假象，少数来自搜索类算法的停止条件。它提醒我们：加速比的分子分母必须可比。而”放大问题”这句话本身含义模糊，这直接引出了本章之后要讨论的扩展性分类。

## 4 强扩展与弱扩展

### 4.1 两种术语

讲者指出业界，尤其是超级计算领域，有两种讨论问题规模扩展的方式。

强扩展是经典的固定问题：拿同一个问题 $X$，看它在 $P$ 个处理器上比在 1 个处理器上快多少，取倒数就是加速比。固定问题，变化资源。

弱扩展则不同：随着资源增长，同时让问题规模成比例增长，然后看效率能多接近 1.0。也就是说，能否用 $P$ 倍的资源在同样时间内解决 $P$ 倍大的问题。

讲者指出后者其实更贴近真实应用想做的事。

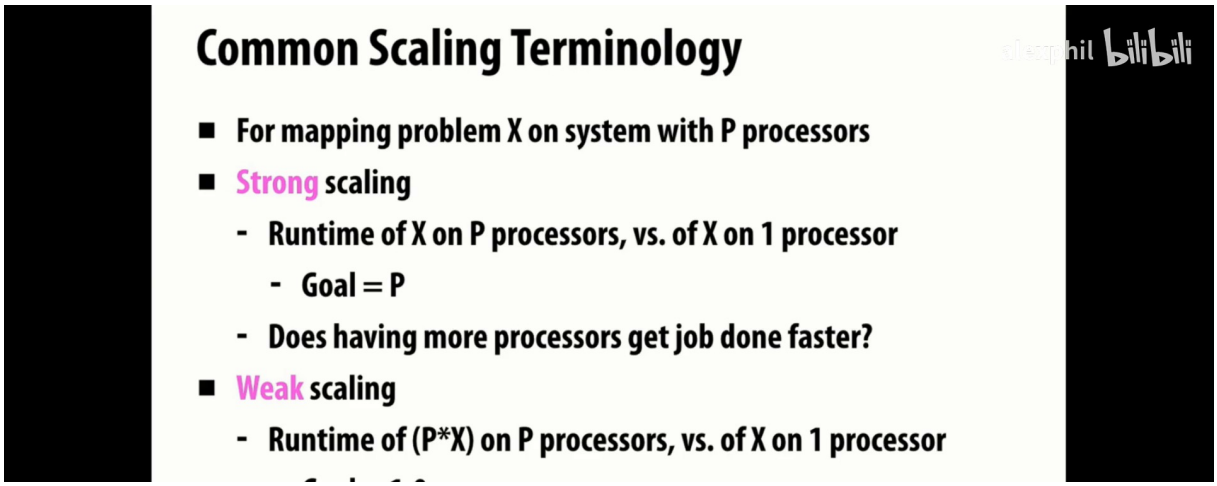



图 7: 常见扩展性术语：强扩展与弱扩展[7]

## 4.2 看懂架构的可扩展设计

讲者提到架构师也在谈扩展性，但他们的问题不同：一种架构能否靠复制单元覆盖从手机到集群的整个产品线。

他给出的例子：几百美元就能买到一块包含两个这类核心加一个 ARM 作为主机的片上系统，装在小盒子里，广泛用于移动设备。而 GHC 机器里那台是个庞然大物——20 个这样的核心，功耗高、价格贵。从设计角度看，这意味着厂商可以通过复制同一个单元派生出整个产品谱系，这是可扩展设计的好例子。

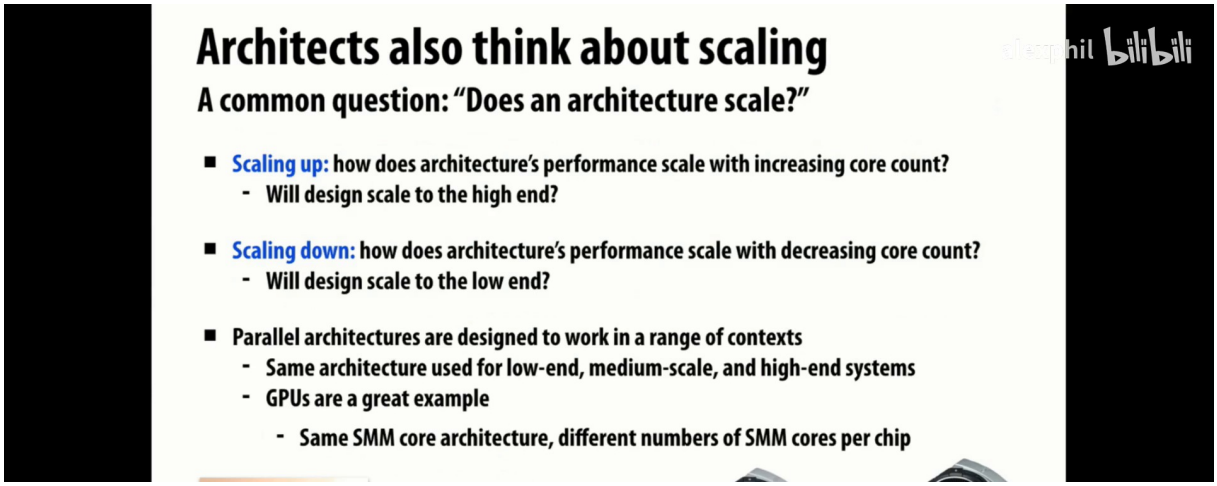



图 8: 架构师也考虑扩展性：能否靠复制单元覆盖不同规模的产品[8]

[7] 视频画面时间区间：00:14:30–00:14:45。若为裁剪图，时间区间仍对应原视频画面。
[8] 视频画面时间区间：00:12:00–00:12:15。若为裁剪图，时间区间仍对应原视频画面。

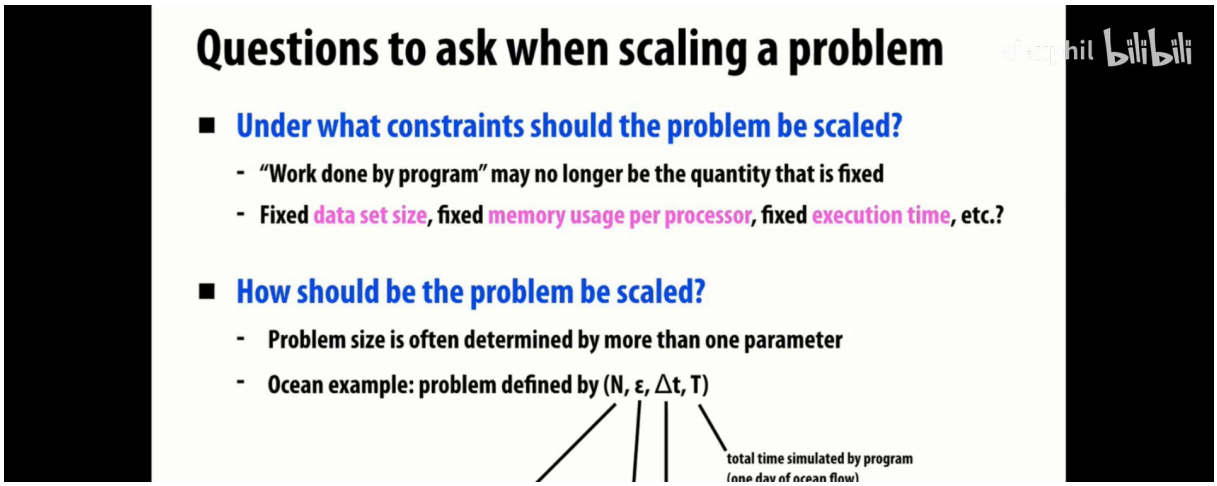



图 9: 扩展一个问题时需要先问的问题[9]

### 4.3 本章小结

强扩展固定问题、增加资源，衡量的是加速比；弱扩展让问题与资源同比增加，衡量的是效率接近 1 的程度。硬件层面的扩展性是另一回事——指同一设计单元复制后能覆盖多大范围的产品。

术语确定后，接下来的问题是：这三种衡量方式在真实的网格求解器上分别表现如何？

## 5 三种约束下的定量分析

这是本讲的核心。讲者把扩展目标分成三类，并用同一份网格求解器推出各自的通信计算比。

### 5.1 三维参数：扩展本身是多变量的

在进入分类之前，讲者先说明问题的复杂程度。以海洋模拟为例，参数包括：$N \times N$ 的网格、收敛误差容限 $\varepsilon$、时间步长 $\Delta t$、以及要模拟的总时长 $T$。

这些参数彼此纠缠。相对独立的是 $T$——跑 10 年还是 100 年的洋流，这一项对性能的影响大致线性。

讲者的结论是：一旦深入某个应用去问” 我该怎么扩展它”，就会发现这是个多元问题。而即使确定了各参数，把它们折算成一个” 问题规模” 的度量也不那么直接。

**两类视角回答的是不同问题**

应用导向问的是” 这门科学想把问题做多大”，答案由物理模型与精度要求决定。资源导向问的是” 这台机器能让我做多大”，答案由内存与时间预算决定。前者设定目标，后者设定边界，实际可解的问题落在两者的交集里。

他因此区分两种视角：应用导向的扩展关心具体参数怎么变；资源导向的扩展则从机器约束出发。

[9] 视频画面时间区间：00:16:00–00:16:15。若为裁剪图，时间区间仍对应原视频画面。

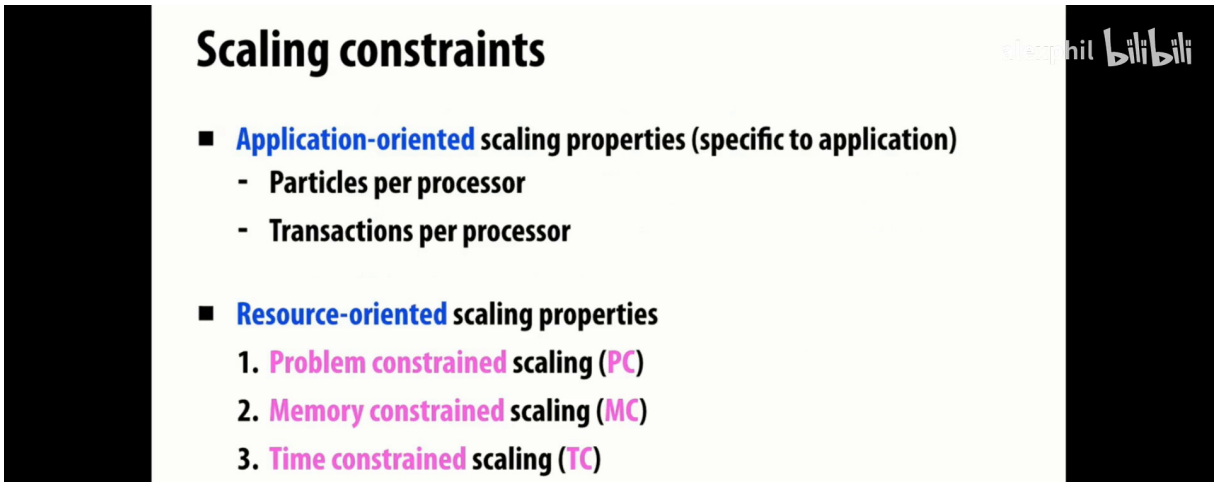



图 10: 扩展约束：应用导向与资源导向两类属性[10]

### 5.2 约束一：问题约束（经典加速比）

问题约束就是固定问题、增加资源，也就是强扩展。

在网格求解器上，每个处理器每一步的计算量是 $N^2/P$，通信量正比于 $N/\sqrt{P}$。因此通信与计算之比为：

$$\left(\frac{C}{W}\right)_{\text{problem}} = \frac{N/\sqrt{P}}{N^2/P} \propto \frac{1}{\sqrt{P}}$$

- $N$：网格边长，问题固定意味着 $N$ 不变；
- $P$：处理器数；
- $N^2/P$：每处理器每步的计算量；
- $N/\sqrt{P}$：每处理器每步的通信量，正比于块周长。

坏消息很明确：$P$ 增大时这个比值按 $1/\sqrt{P}$ 恶化，计算越来越被通信压住，于是 $P$ 的扩展能力被限制住。

[10] 视频画面时间区间：00:17:30–00:17:45。若为裁剪图，时间区间仍对应原视频画面。

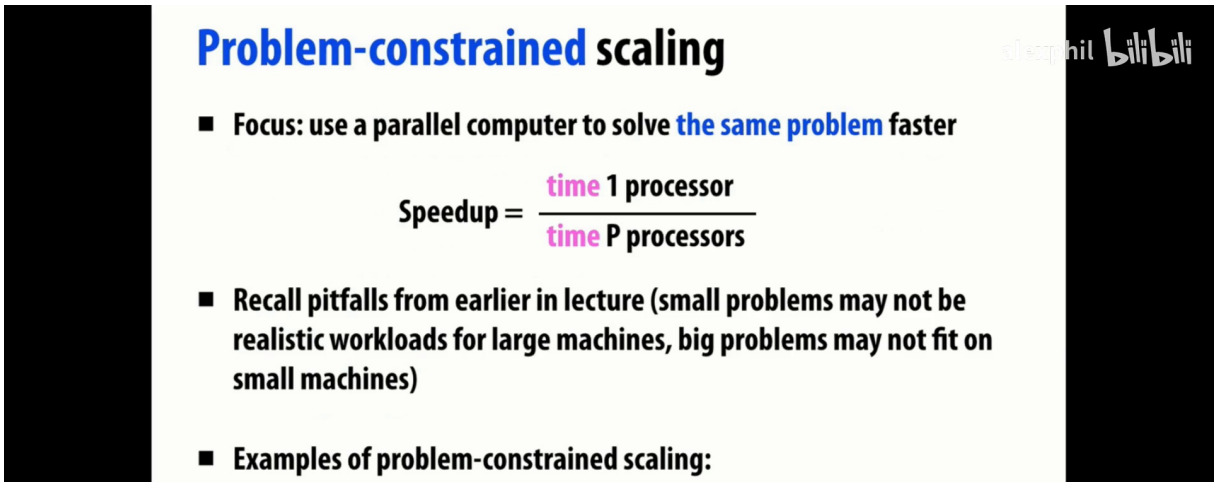



图 11：问题约束扩展：用并行系统更快地解决同一个问题[11]

### 5.3 约束二：时间约束

时间约束的目标是：从 1 个处理器换到 $P$ 个，希望解决尽可能大的问题而总时间不变。

讲者先假设线性加速，即算力变为 $P$ 倍。那么问题规模能放大多少？关键细节在于：网格边长变大时，收敛所需的迭代次数也会增加，因为达到稳定状态的传播时间正比于网格直径。

于是问题规模的两个维度都要按 $P^{1/3}$ 增长：

$$N' = N \cdot P^{1/3}, \qquad I' = I \cdot P^{1/3}$$

- $N'$：新的网格边长，按 $P^{1/3}$ 放大，即指数为 0.333；
- $I'$：新的迭代次数，同样按 $P^{1/3}$ 放大；
- 两者相乘使总计算量变为 $P^{2/3} \cdot P^{1/3} = P$ 倍，正好用掉 $P$ 倍算力。

代回去求通信计算比：

$$\left(\frac{C}{W}\right)_{\text{time}} \propto P^{-1/6} \approx P^{-0.167}$$

讲者说这仍然随 $P$ 恶化，但比问题约束好得多——因为我们在放大网格的同时也做了更多时间步，这一项对我们有利。

[11] 视频画面时间区间：00:18:30–00:18:45。若为裁剪图，时间区间仍对应原视频画面。

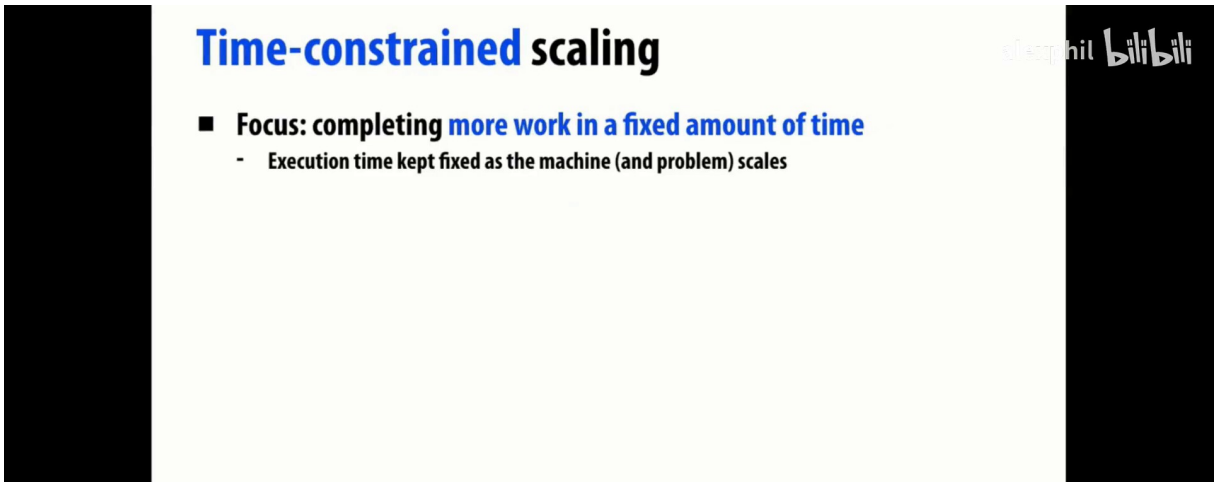



图 12: 时间约束扩展：在固定时间内完成更多工作[12]

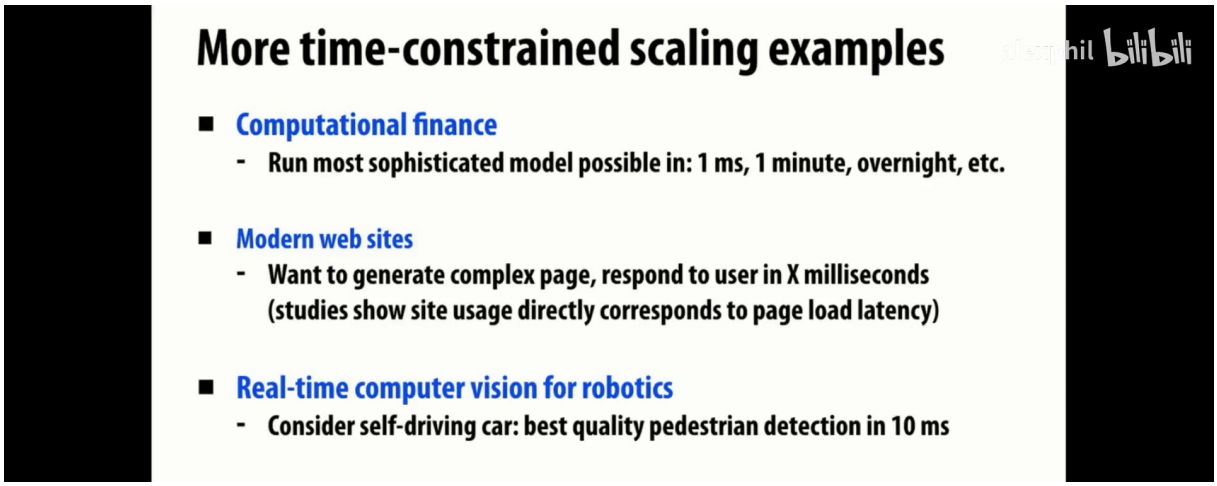



图 13: 时间约束扩展的更多例子：如计算金融中运行更精细的模型[13]

### 5.4 约束三：内存约束

内存约束的目标是：既然有了 $P$ 倍的处理器，就有了 $P$ 倍的内存，那么装下能装的最大问题并跑完它。

总运行时间会更长，但能解决的问题比原来大。关键在于这里的扩展方式：网格边长按 $P^{1/2}$ 增长，使得每处理器的元素数保持不变。

$$\left(\frac{C}{W}\right)_{\text{memory}} = \text{常数}$$

- 每处理器的数据量随 $P$ 增长而保持同比例增长；
- 于是每处理器的计算量与通信量之比不变；
- 通信计算比不随 $P$ 恶化。

[12] 视频画面时间区间：00:20:00–00:20:15。若为裁剪图，时间区间仍对应原视频画面。

[13] 视频画面时间区间：00:24:00–00:24:15。若为裁剪图，时间区间仍对应原视频画面。

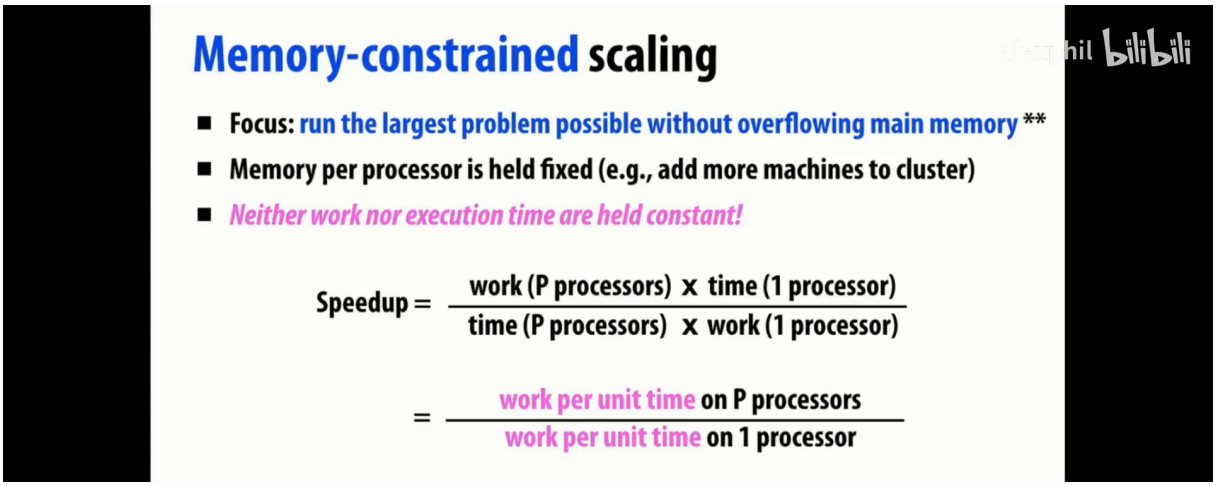



图 14: 内存约束扩展：在内存装得下的前提下运行最大的问题[14]

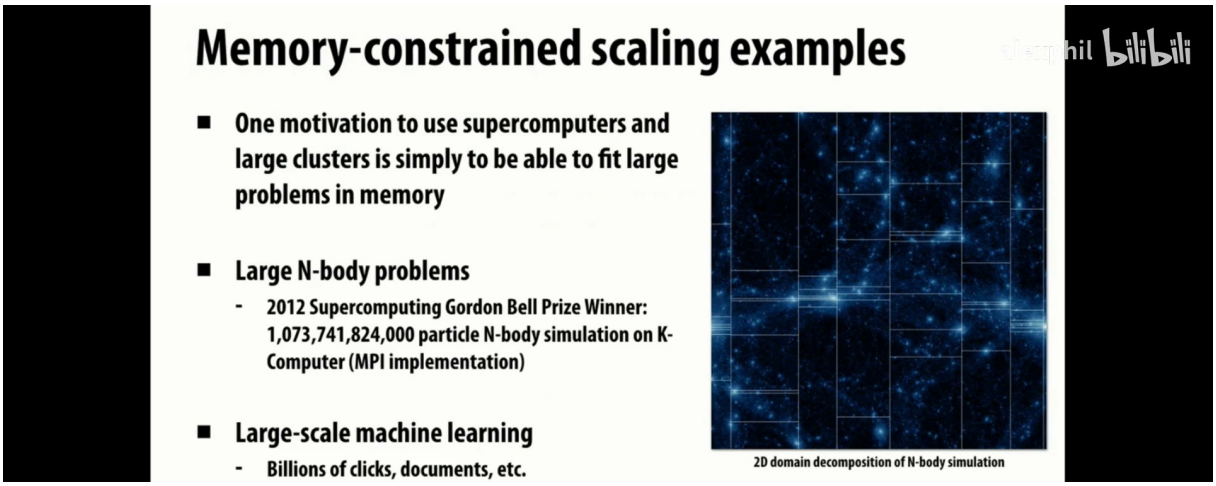



图 15: 内存约束扩展的例子：这是使用超级计算机的主要动机之一[15]

## 5.5 三种约束的排序

讲者给出结论性的排序。

**内存约束下效率稳定的几何原因**

按内存线性放大时，每处理器分到的元素数保持不变，因此块的面积与周长之比保持不变。通信正比周长、计算正比面积，比值自然恒定。这也说明为什么超算的采购逻辑强调内存墙——扩内存往往比扩算力更能保持并行效率。

内存约束最容易满足：如果原来就是好的，这样扩展会很顺畅。

时间约束次之：仍然会变差，但速率温和。

问题约束最难：也就是经典的加速比，是最难达成扩展目标的一种。

[14] 视频画面时间区间：00:25:30–00:25:45。若为裁剪图，时间区间仍对应原视频画面。

[15] 视频画面时间区间：00:26:30–00:26:45。若为裁剪图，时间区间仍对应原视频画面。

| 约束类型 | 问题规模如何变 | 通信计算比 | 难度 |
|---|---|---|---|
| 问题约束 | 不变 | $\propto P^{-1/2}$ | 最难 |
| 时间约束 | 边长与迭代各按 $P^{1/3}$ | $\propto P^{-1/6}$ | 中等 |
| 内存约束 | 按内存线性放大 | 不变 | 最易 |

表 1: 三种扩展约束下的通信计算比随处理器数的变化

**扩展性结论取决于你问的是哪个问题**

同一份代码、同一台机器，按问题约束衡量可能”扩展性很差”，按内存约束衡量却”扩展性很好”。这两句话并不矛盾，因为它们问的是不同的问题。因此在评价一个系统的扩展性之前，必须先说明采用哪种约束，否则数字本身没有意义。

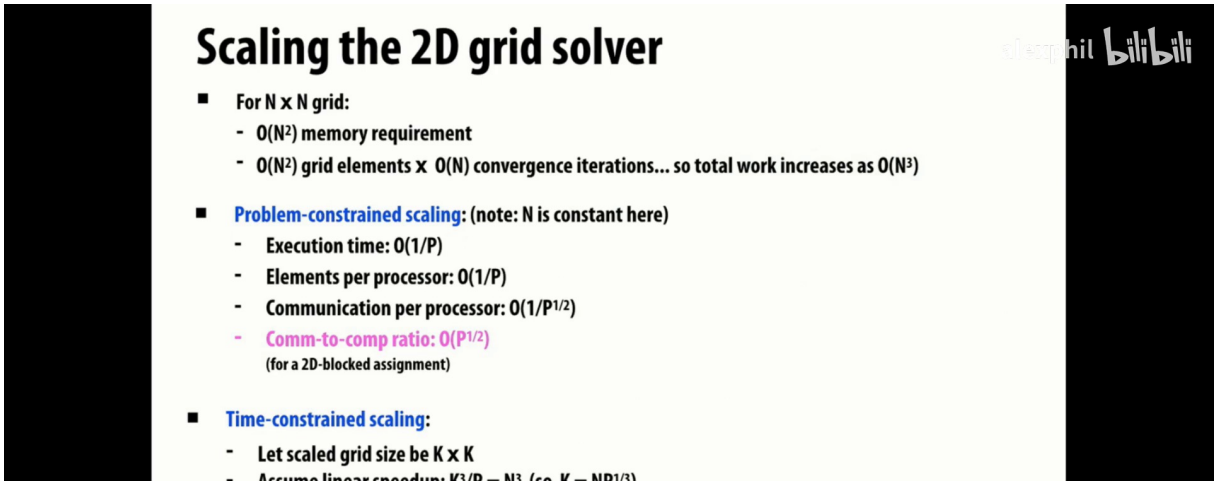



图 16: 扩展二维网格求解器：三种约束下通信计算比的推导[16]

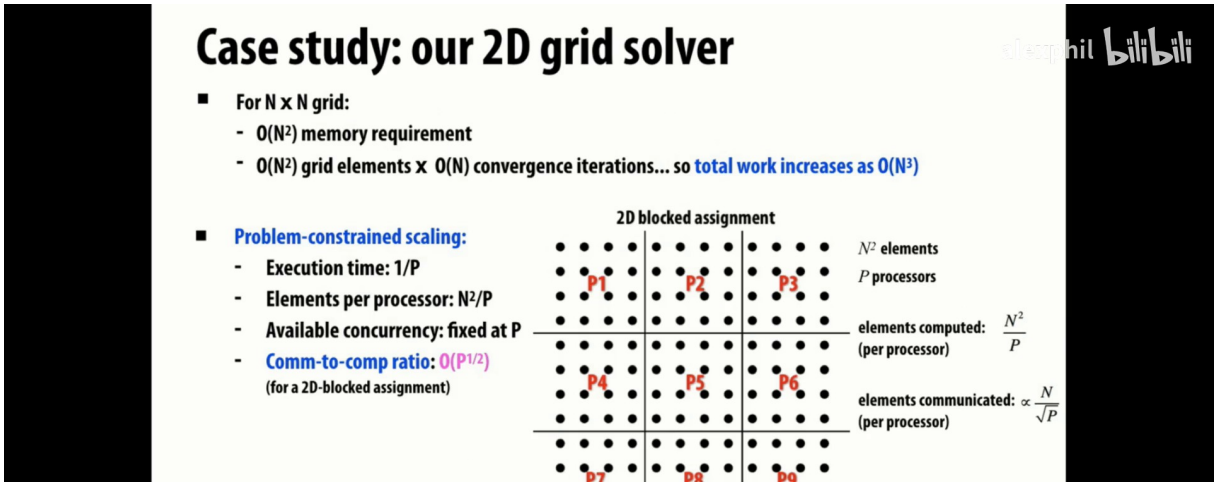



图 17: 案例研究：二维网格求解器的内存与元素复杂度[17]

[16] 视频画面时间区间：00:33:00–00:33:15。若为裁剪图，时间区间仍对应原视频画面。

[17] 视频画面时间区间：00:31:00–00:31:15。若为裁剪图，时间区间仍对应原视频画面。

### 5.6 本章小结

问题约束下通信计算比按 $P^{-1/2}$ 恶化，时间约束下按 $P^{-1/6}$ 温和恶化，内存约束下保持不变。难易顺序依次是问题约束最难、时间约束次之、内存约束最易。这说明”扩展性问题”的答案取决于选了哪种约束——离开约束谈扩展性没有意义。

讲者随后提醒，真实情形比这张表更复杂，因为各参数会互相牵动，而实际的使用场景也各不相同。

## 6 真实场景中的扩展与模拟方法

### 6.1 同一应用，不同约束

讲者用 Pixar 的例子说明这一点。渲染一个”镜头”（即电影中的一段连续帧）时，管线前面各阶段的约束不同，而最后有一遍大规模渲染把整段镜头算出来——那一步受各种因素共同限制。

他的论点不是关于 Pixar 本身，而是一般性的：即使在同一类应用内部，不同的使用场景下也会有不同的约束。

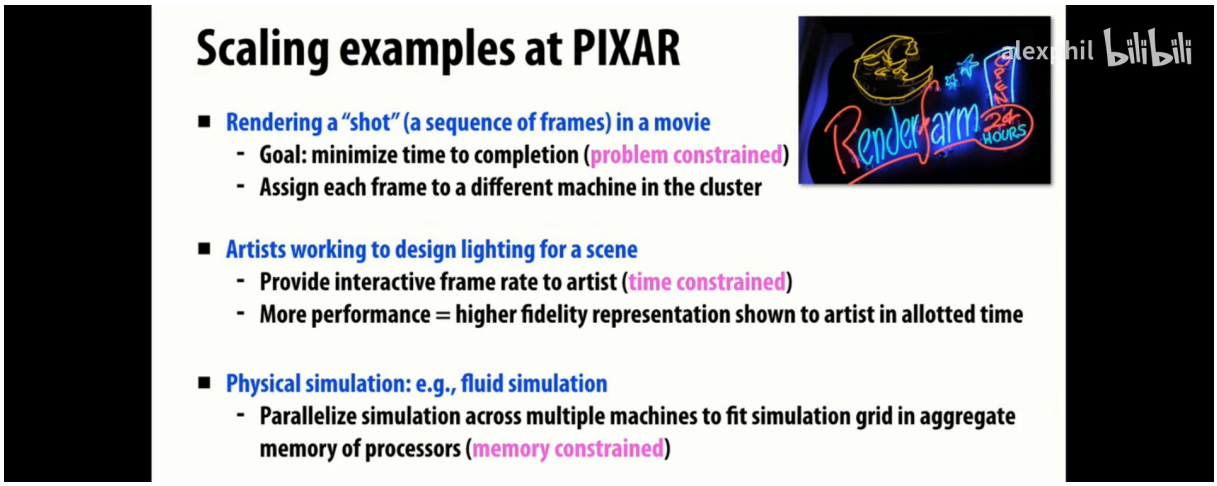



图 18: Pixar 的扩展性案例：渲染一个镜头时各阶段的约束不同[18]

[18] 视频画面时间区间：00:27:30–00:27:45。若为裁剪图，时间区间仍对应原视频画面。

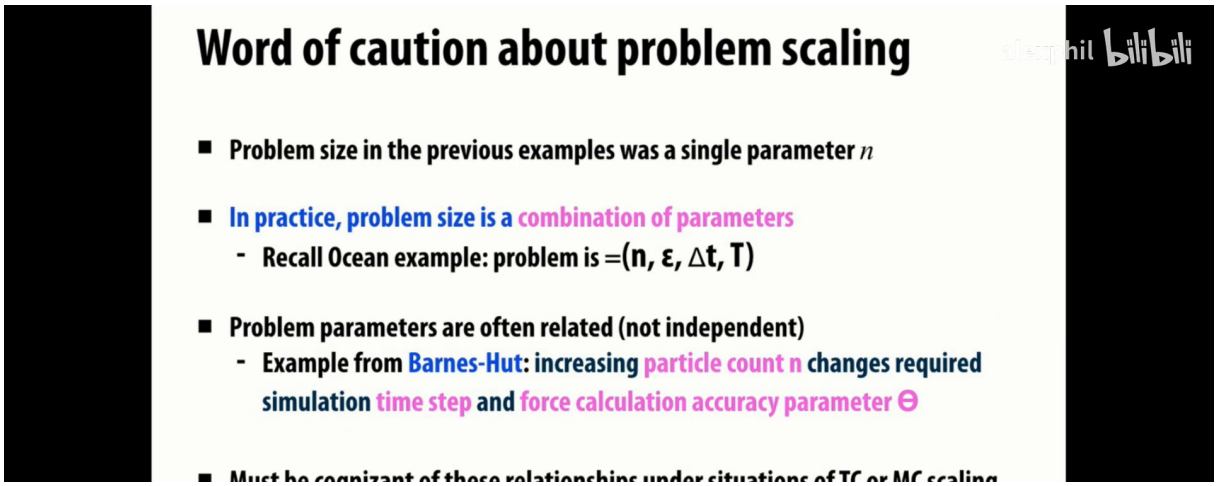



图 19: 关于问题扩展的注意事项：前例中的问题规模只是单项参数[19]

### 6.2 本章小结

同一应用在不同使用场景下有不同的约束，Pixar 的渲染管线就是一例：前面各阶段与最后的整段渲染，瓶颈并不相同。因此扩展性分析必须绑定具体场景，而非笼统地谈论应用。

当要评估的是尚未存在的硬件时，只能依靠模拟。这是下一章的主题。

## 7 用模拟评估架构

### 7.1 为什么要靠模拟

讲者转向另一个话题：架构师怎么评估一个新的设计想法。答案是模拟——因为你不能为了评估就把芯片造出来。

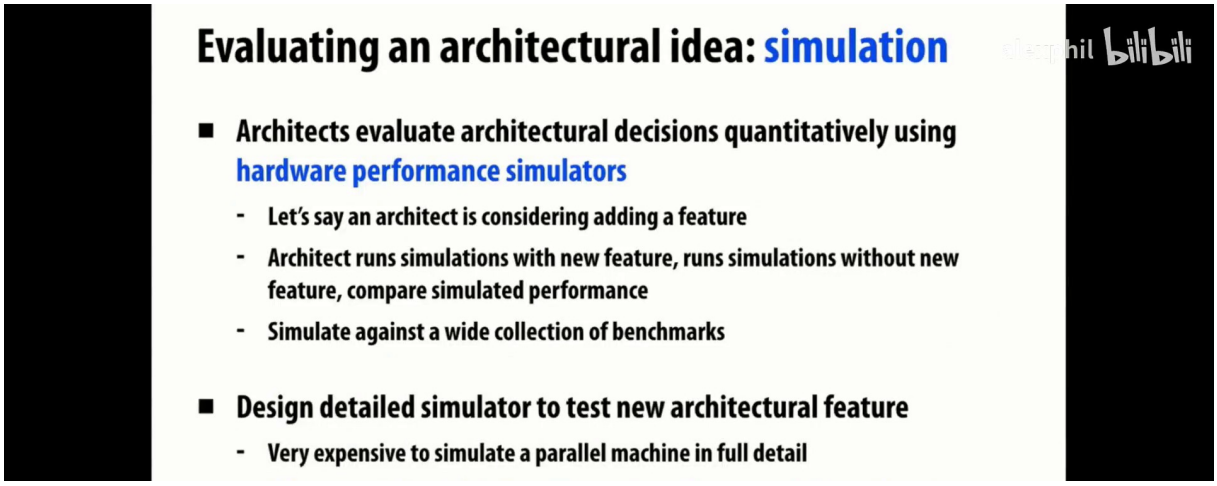



图 20: 评估架构想法：架构师靠模拟来量化设计决策[20]

有两种主要做法。

[19] 视频画面时间区间：00:37:00–00:37:15。若为裁剪图，时间区间仍对应原视频画面。

[20] 视频画面时间区间：00:38:00–00:38:15。若为裁剪图，时间区间仍对应原视频画面。

**两种模拟方法的保真度与成本互换**

跟踪驱动把”程序行为”固定在轨迹里，模拟器只需处理存储层次，因此单次运行快、可扫描大量架构参数。执行驱动需要完整模拟指令流，能捕捉程序对架构的适应，但慢几个数量级。选择哪一种取决于要评估的改动是否会影响程序自身的行为。

跟踪驱动：在真实机器上运行真实代码，记录下所有内存访问等事件的轨迹，然后在模拟器里重放这条轨迹。它不需要真的建模执行，因此快。

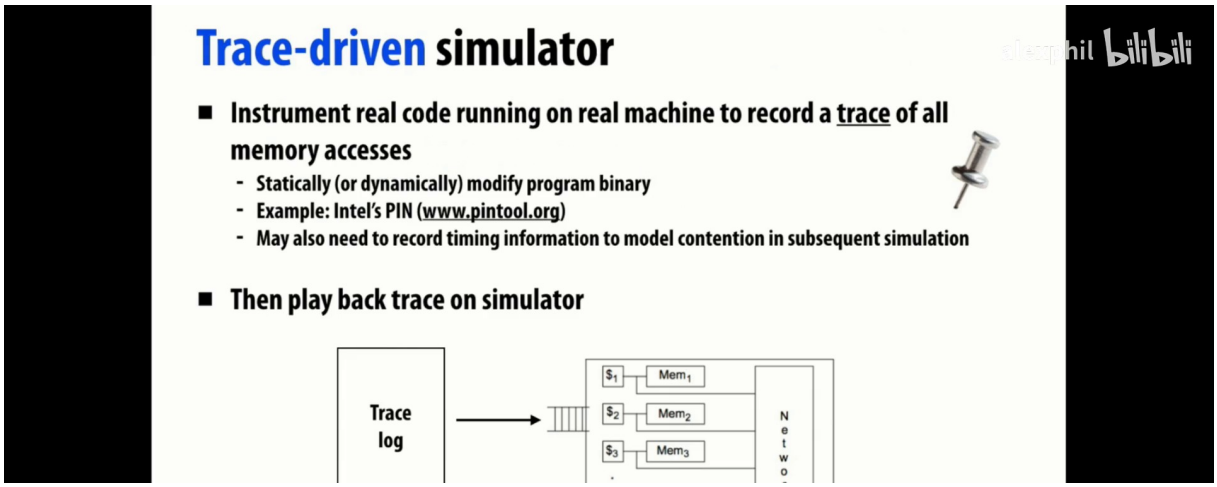



图 21：跟踪驱动的模拟器：对真实代码插桩以记录访问轨迹[21]

**评估结论的偏差方向要能预判**

知道跟踪驱动会低估新硬件收益，就能解释为什么某些架构改动在模拟中收益平平、在真实原型上却效果显著。反之，若模拟显示巨大收益而实测没有，也应怀疑轨迹没有反映新架构带来的访问模式变化。

执行驱动：直接模拟执行，包含系统的反馈——系统会动态适应计算的实际执行情况，因此不同反馈会引出不同轨迹。它能处理更多情形，但更难构建、运行更贵。

## 7.2 跟踪驱动的两个风险

讲者花了相当篇幅讨论跟踪驱动方法的隐患，这部分对做性能评估的人很有价值。

第一个风险是过拟合。如果反复用同一组轨迹调优，很容易写成专门照顾这些特例的代码。他的建议是理想情况下每次都重新采样问题空间，生成新的样本集——但现实中往往做不到。

第二个风险是开放环路假设。轨迹方法默认新系统中的流量与旧系统完全相同。但在时间约束扩展中，我们希望用同样的时间做更复杂的计算，那么真实轨迹代表的只是固定问题，未必能代表放大后的问题。

[21] 视频画面时间区间：00:39:30–00:39:45。若为裁剪图，时间区间仍对应原视频画面。

**用回放轨迹评估新系统会系统性高估收益**

轨迹固定了访问模式，而新架构往往会改变访问模式（更大的缓存减少了缺失、更快的互连改变了同步频率）。因此在旧机器上采集的轨迹，用来预测新机器的表现时，会低估新硬件缓解访存压力的效果。执行驱动模拟能部分修正这一点，代价是构建成本。

### 7.3 本章小结

真实应用中不同使用场景对应不同约束，Pixar 的渲染管线就是一例。架构评估依赖模拟，跟踪驱动快但有开放环路与过拟合的风险，执行驱动能反馈但更贵。这些方法问题与前面的扩展性分析是同一件事的两面：都在追问”用什么条件下的数字来代表性能”。

讲者的收尾建议因此很务实：不必靠随机猜测与试错，可以让真实数字驱动决策。

## 8 总结与延伸

讲者收尾时说，做了这么多工作，重点是你不必靠随机猜测和试错去看发生了什么，可以更有原则、由真实数字驱动地做决定。

把全讲串起来，是一次对”性能”这个概念的拆解。加速比有两个陷阱：基线若用带并行开销的伪串行版本会高估收益；问题太小会让每核喂不饱而低估收益。超线性加速多数是缓存合并的假象，少数来自搜索类算法的停止条件，它提醒我们基线必须可比。要问”扩展得好不好”，先得区分强扩展（固定问题）与弱扩展（问题同比放大），而更实用的分法是按约束：问题约束下通信计算比按 $P^{-1/2}$ 恶化，时间约束下按 $P^{-1/6}$ 温和恶化，内存约束下保持不变——所以内存约束最容易、问题约束最难。同一份代码在不同约束下可以同时”扩展性很差”与”扩展性很好”。评估架构则要用模拟，跟踪驱动快但有开放环路与过拟合风险。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 加速比不是性能的同义词。它衡量的只是”相对于某个基线快多少”，而这个基线的选择本身就是一个需要论证的决定。报告中给出加速比时，必须同时说明基线是什么、问题规模多大。
- 三种约束的排序有几何原因。通信正比边界、计算正比面积，因此只有让每处理器的数据量随 $P$ 同比增长（内存约束），比值才能保持稳定。其余两种都在稀释这个比值。
- 超线性是诊断信号而非成绩。看到它首先该检查基线是否装不进缓存，而不是宣称实现优越。真实存在的例外（搜索类）有明确的机制解释：任一分支成功即完成。
- 模拟方法的偏差方向可以预判。跟踪驱动会系统性低估新硬件的收益，因为轨迹固化了旧的访问模式。知道偏差方向，就能判断结论应当偏向哪一侧。

这堂课的边界在于它讨论的是测量与评估的方法论，不涉及具体应用的调优。讲者明确表示后续作业会让学生自己面对这些选择。

两个可以直接从本讲数据提出的下一步问题：其一，既然问题约束最难，那么在必须做固定问题加速的场合（例如实时渲染有硬性帧率要求），唯一出路是否只剩提高算术强度——也就是改变算法而非改进实现？其二，跟踪驱动假设开放环路，那么当被评估的硬件改动会影响程序行为（例如更强的缓存导致线程调度改变）时，什么条件下回放仍然可信？